

**STANDAR OPERASIONAL PROSEDUR
PENGUJIAN API
OLEH QUALITY ASSURANCE**



**SOFTWARE QUALITY
PT NEURONWORKS INDONESIA
2025**

DAFTAR ISI

1. PENDAHULUAN	4
1.1. Latar Belakang.....	4
1.2. Tujuan	4
1.3. Ruang Lingkup.....	4
1.4. Sasaran Pembaca	5
1.5. Struktur Dokumen	5
2. STANDAR AKSES QA	6
2.1. Standar Akses VPN.....	6
2.1.1. Standar Pengujian Pembuatan VPN untuk QA	6
2.1.2. Standar Pengujian Penetrasi VPN	6
2.2. Standar Akses Gitlab	6
2.2.1. Standar Pengujian Pembuatan Akun Gitlab untuk QA	6
2.3. Standar Akses Test Case.....	6
2.3.1. Standar Pengujian Akses Dokumen Test-Case	7
3. STANDAR AKUN PENGUJIAN QA	8
3.1. Standar Role Akun Pengujian.....	8
3.2. Standar Hak Akses Akun Pengujian.....	8
4. STANDAR KLASIFIKASI KASUS DAN ALUR PENGUJIAN	1
4.1. Standar Umum Pengujian API.....	1
5. STANDAR KLASIFIKASI TIPE KASUS PENGUJIAN	3
5.1. Standar Klasifikasi Tipe Pengujian Positif.....	3
5.2. Standar Klasifikasi Tipe Pengujian Negatif	3
5.3. Standar Klasifikasi Tipe Pengujian Security.....	5
6. STANDAR PENGUJIAN BERBASIS SCENARIO.....	10
6.1. Prosedur Dasar.....	10
6.1.1. HTTP Methods: Mengirim API	10
6.1.2. HTTP Status Code: Response dari API.....	10
6.1.3. Authentication & Authorization: Siapa Saja Yang Boleh Masuk	11
6.1.4. Headers: Informasi Tambahan di Request	11
6.1.5. Path & Query Parameters: Memberikan Instruksi ke API.....	11
6.1.6. Versioning: API Jangan Sampai Bentrok!	11
6.1.7. Pagination: Agar API Tidak Kelebihan Beban	12
6.1.8. Input Payload Validation: Data Masuk Harus Benar	12

6.1.9.	Response Payload Validation: Data Keluar Harus Benar.....	12
6.1.10.	Error Handling: Supaya API Tidak Malfuntion	13
7.	STANDAR PENGUJIAN MENGGUNAKAN POSTMAN	13
7.1.	GET	13
7.2.	POST	14
7.3.	PUT	15
7.4.	DELETE	16
8.	STANDAR PENGUJIAN AUTOMATION MENGGUNAKAN NEWMAN	18
8.1.	Install Newman	18
8.2.	SetUp Newman: Reporting dengan Allure	19
8.3.	Implementation	21
8.3.1.	Membuat Collection Sesuai Standar	21
8.4.	Execution.....	25
8.4.1.	Menjalankan Report Newman di HTML	25
8.4.2.	Menjalankan Newman di Terminal	25
8.4.3.	Menjalankan Newman di Allure	26

1. PENDAHULUAN

1.1. Latar Belakang

Standar Operasional Prosedur merupakan acuan dalam melaksanakan tugas atau pekerjaan sesuai fungsi alat penilaian kinerja pada instansi, organisasi atau sistem. Tanpa adanya Standar Operasional Prosedur maka organisasi atau sistem tidak bisa berjalan dengan semestinya. Standar Operasional Prosedur bertujuan untuk mencapai visi misi yang sudah telah diatur, dan juga dapat membantu proses perkembangan pada organisasi atau sistem dalam upaya untuk memastikan kualitas perangkat lunak yang optimal. Standar pengujian QA (*Quality Assurance*) menjadi elemen kunci dalam proses pengembangan aplikasi, standarisasi ini dirancang untuk memastikan bahwa setiap aplikasi yang dikembangkan memenuhi persyaratan fungsional, keamanan, dan kinerja sebelum diimplementasikan di lingkungan produksi (*Environment Production*).

1.2. Tujuan

Dokumen ini disusun sebagai acuan proses SOP yang memberikan panduan lengkap dan teststruktur mengenai standar-standar yang harus diikuti oleh Tim QA dalam melakukan pengujian terhadap aplikasi berbasis web dan mobile. Standar ini berisi SOP yang mencakup berbagai aspek mulai dari akses QA, akun pengujian, klasifikasi kasus pengujian, hingga pelaporan hasil pengujian. Dengan mengikuti standar ini, diharapkan proses pengujian dapat dilakukan secara konsisten dan efisien, serta menghasilkan produk perangkat lunak yang berkualitas tinggi.

1.3. Ruang Lingkup

Dokumen ini mencakup standar operasional prosedur atau pengujian yang perlu dipatuhi dalam proses pengujian perangkat lunak yang terbagi menjadi beberapa bagian utama:

1. Standar akses QA: Menguraikan prosedur dan persyaratan terkait akses VPN, GitLab dan dokumen test-case yang diperlukan oleh tim QA.
2. Standar Akun Pengujian QA: Menjelaskan standar terkait peran, hak akses, dan fitur-fitur yang diperlukan untuk akun pengujian yang digunakan oleh QA.
3. Standar Klasifikasi Kasus dan Alur Pengujian: Memberikan panduan mengenai klasifikasi kasus pengujian (positif dan negatif) serta alur pengujian yang harus diikuti.
4. Standar Pengujian berbasis scenario: Memberikan prosedur yang berisi uji scenario, data uji, dan prosedur besar pengujian API Test
5. Standar Uji Coba Pertama: Memberikan panduan mengenai uji coba pertama sesuai SOP API Testing

1.4. Sasaran Pembaca

Dokumen ini ditujukan bagi seluruh anggota Tim QA, *Developer*, *Project Manager* serta pihak-pihak terkait yang terlibat dalam pengembangan dan pengujian perangkat lunak di PT Neuronworks Indonesia. Dengan mengikuti panduan yang ada dalam dokumen ini, diharapkan setiap individu dalam tim dapat berkontribusi secara efektif dalam memastikan kualitas aplikasi yang dihasilkan.

1.5. Struktur Dokumen

Dokumen ini disusun dengan struktur yang jelas dan terperinci sebagai berikut:

- a. Standar Akses QA: Mengatur standar akses terkait VPN, GitLab, dan dokumen test-case.
- b. Standar Akun Pengujian QA: Menjelaskan standar peran, hak akses, dan fitur akun pengujian.
- c. Standar Klasifikasi Kasus dan Alur Pengujian: Menguraikan standar klasifikasi kasus pengujian dan alur pengujian
- d. Standar Pengujian berbasis scenario: Memberikan prosedur yang berisi uji scenario, data uji, dan prosedur besar pengujian API Test
- e. Standar Pengujian menggunakan Postman: Memberikan panduan mengenai uji coba melalui postman

2. STANDAR AKSES QA

Dalam proses pengujian ada beberapa akses yang perlu dimiliki oleh QA dan standar akses mendefinisikan bagaimana mekanisme QA dalam mendapatkan akses tersebut sesuai dengan standar yang telah ditetapkan. Standar ini mencakup mekanisme pengadaan dan penggunaan akses VPN, GitLab, serta dokumen Test-Case, yang semuanya penting untuk memastikan kelancaran dan keamanan proses pengujian.

2.1. Standar Akses VPN

Akses VPN merupakan elemen penting dalam proses pengujian oleh QA. Akses VPN diperlukan untuk dapat mengakses repositori GitLab atau server aplikasi dan sistem yang sedang dikembangkan oleh tim development, QA memerlukan VPN yang disediakan oleh klien perusahaan sesuai dengan ketentuan yang berlaku dari pihak klien.

2.1.1. Standar Pengajuan Pembuatan VPN untuk QA

Pengajuan akses VPN akan dilakukan oleh Project Manager dari tim tempat QA ditempatkan. QA perlu menyediakan data pendukung yang diperlukan, termasuk data diri, KTP, dan tanda tangan pakta integritas, untuk melengkapi formulir pengajuan. Setelah data terkumpul, Project Manager akan memproses pengajuan dan mengoordinasikan dengan tim administrasi di perusahaan untuk memastikan akses VPN diberikan secara tepat dan aman.

2.2. Standar Akses Gitlab

Akses gitlab merupakan salah satu elemen penting dalam proses pengujian oleh QA. Karena setiap hasil pembuatan pengujian, pelaporan pengujian serta dokumen hasil pengujian harus disertakan kedalam issue yang ada pada gitlab.

2.2.1. Standar Pengajuan Pembuatan Akun Gitlab untuk QA

Pengajuan akses VPN akan dilakukan oleh Project Manager dari tim tempat QA ditempatkan. Umumnya pengajuan pembuatan akun gitlab untuk QA akan diajukan bersamaan dengan pengajuan pembuatan akses VPN. Karena gitlab klien hanya akan dapat diakses ketika QA sudah bisa terhubung dengan VPN dari klien.

2.3. Standar Akses Test Case

Akses Test Case merupakan salah satu elemen penting dalam proses pengujian oleh QA. Karena setiap tahapan pengujian, hasil pengujian dan catatan lainnya selama pengujian akan disertakan didalam dokumen Test Case tersebut.

2.3.1. Standar Pengajuan Akses Dokumen Test-Case

Dokumen Test Case telah disediakan oleh tim Software Quality dan dapat diakses oleh QA melalui pengajuan akses kepada tim Software Quality. Pengajuan harus mencantumkan nama aplikasi yang membutuhkan akses ke dokumen Test Case. Untuk mengakses dokumen tersebut, QA memerlukan alamat email agar link akses dapat dikirimkan.

3. STANDAR AKUN PENGUJIAN QA

Dalam proses pengujian, QA memerlukan akun pengujian yang sesuai dengan kebutuhan dan konfigurasi yang telah ditetapkan. Standar ini mengatur bagaimana akun pengujian disiapkan dan digunakan untuk memastikan pengujian berjalan dengan efektif dan efisien.

3.1. Standar Role Akun Pengujian

Akun pengujian yang digunakan oleh QA harus memiliki role yang sesuai dengan jenis pengujian yang dilakukan. Untuk memastikan bahwa setiap perubahan data yang terjadi pada sistem dapat diidentifikasi dengan tepat, QA harus menggunakan akun pengujian yang terpisah dari akun dengan role lainnya. Salah satu caranya adalah dengan membuat akun **khusus** untuk QA dengan role atau peran sebagai Tester.

3.2. Standar Hak Akses Akun Pengujian

Akun QA harus memiliki hak akses yang mencakup semua role akun dalam sistem untuk memastikan pengujian dapat dilakukan secara menyeluruh. Hak akses ini mencakup akses ke semua menu dan fungsi dalam aplikasi, sehingga QA dapat memverifikasi semua fitur dan skenario pengujian tanpa batasan. Dengan demikian, QA dapat melakukan pengujian secara komprehensif untuk memastikan kualitas sistem secara keseluruhan

4. STANDAR KLASIFIKASI KASUS DAN ALUR PENGUJIAN

Dalam pengujian perangkat lunak, penting untuk mengklasifikasikan kasus pengujian secara efektif untuk memastikan bahwa seluruh aspek dari sistem diuji dengan baik. Klasifikasi kasus pengujian membantu dalam merencanakan dan mengorganisir pengujian, memastikan bahwa berbagai skenario pengujian diterapkan untuk mencapai hasil yang komprehensif dan akurat.

4.1. Standar Umum Pengujian API

Pengujian API dapat menjadi salah satu bagian yang paling menantang dalam pengujian perangkat lunak dan QA karena API dapat menjadi rumit, API sering kali didasarkan pada protokol dan standar yang sering tidak bisa ditemukan dalam jenis pengujian lainnya. Pada tingkat paling dasar, pengujian API dimaksudkan untuk mengungkap bug: ketidakkonsistenan atau penyimpangan dari perilaku yang diharapkan. Pengujian berkelanjutan juga sangat penting untuk memastikan API terus berfungsi saat publik memiliki akses ke API tersebut. Risiko menempatkan produk yang buruk dan berpotensi tidak aman di pasaran lebih besar daripada biaya untuk mengujinya.

Pengujian API adalah salah satu bagian paling menantang dari rangkaian pengujian perangkat lunak dan QA karena berfungsi untuk memastikan kehidupan digital seseorang berjalan semakin lancar dan efisien. Sementara pengembang cenderung hanya menguji fungsionalitas yang sedang mereka kerjakan, penguji bertugas menguji fungsionalitas individual dan serangkaian atau rantai fungsionalitas, untuk menemukan cara kerja semuanya secara bersama-sama. Bagian pertama pengujian API melibatkan pengaturan lingkungan pengujian, dengan serangkaian parameter yang diperlukan di sekitar API. Ini melibatkan konfigurasi database dan server untuk persyaratan aplikasi. Selanjutnya, buat panggilan API segera untuk memastikan tidak ada yang rusak sebelum Anda melanjutkan untuk memulai pengujian yang lebih menyeluruh. Pengujian otomatis mengharuskan Seseorang menggunakan alat pengujian, seperti SoapUI, sementara pengujian manual terdiri dari penulisan kode Seseorang sendiri untuk menguji API. Pengujian API adalah salah satu area di mana pengujian otomatis sangat direkomendasikan, khususnya di dunia DevOps, pengembangan tangkas, dan siklus pengiriman berkelanjutan.

1) Pengujian API Manual

Saat melakukan pengujian API secara manual, maka perlu diperhatikan hal-hal berikut:

- Pengujian eksplorasi

Menguji API tanpa skenario yang terstruktur untuk menemukan bug yang tidak terduga.

- Pengujian fungsional
Mengevaluasi seberapa mudah API digunakan, dipahami, dan diintegrasikan oleh developer.
- Pengujian ad-hoc
Pengujian tanpa rencana atau dokumentasi khusus untuk menemukan bug secara spontan

2) Pengujian API Otomatis

Saat melakukan pengujian API secara Otomatis, maka perlu diperhatikan hal-hal berikut:

- Pengujian Fungsional API
Memastikan setiap endpoint API bekerja sesuai dengan spesifikasinya
- Pengujian dinamis
Menguji API dalam berbagai kondisi runtime untuk mendeteksi masalah yang muncul saat dijalankan
- Desain pengujian berulang
Menyusun skenario pengujian yang bisa dijalankan berulang kali untuk efisiensi dan konsistensi.
- Menganalisis cakupan pengujian fungsional Seseorang untuk mengetahui apa yang terlewatkan
Memastikan semua fitur API telah diuji dan tidak ada bagian penting yang terlewat.
- Pengujian kesalahan
Menguji bagaimana API menangani input yang salah atau situasi error, termasuk validasi respons dan kode status HTTP.

5. STANDAR KLASIFIKASI TIPE KASUS PENGUJIAN

Pengklasifikasian kasus pengujian melibatkan pembagian kasus menjadi kategori-kategori yang jelas, seperti pengujian positif dan negatif, untuk memastikan bahwa semua aspek fungsionalitas sistem diuji secara menyeluruh.

5.1. Standar Klasifikasi Tipe Pengujian Positif

Pengujian positif dirancang untuk memastikan bahwa sistem berfungsi sesuai dengan harapan ketika diberikan input yang valid dan benar. Untuk dapat melakukan pengujian dengan tipe positif, QA harus mengikuti langkah pengujian yang telah ditentukan dalam dokumen SOP Penggunaan Aplikasi atau Sistem yang sedang dilakukan pengujian. Selain itu, QA harus merujuk pada arahan pengujian yang diberikan oleh Sistem Analis, yang tercatat dalam dokumen Test-Case pada bagian Test Step. Standar ini memastikan bahwa fitur sistem diuji dalam kondisi ideal untuk mengonfirmasi bahwa mereka bekerja sesuai dengan yang diinginkan

5.2. Standar Klasifikasi Tipe Pengujian Negatif

Pengujian negatif bertujuan untuk menguji bagaimana sistem menangani input yang tidak valid atau menguji bagaimana sistem dapat menangani kesalahan. Selain mengikuti arahan dari Sistem Analis yang telah tercantum pada dokumen Test Case pada bagian Test Step tipe pengujian negatif, QA juga dapat mengikuti beberapa panduan berikut dalam mengembangkan pengujian tipe negatif, diantaranya adalah:

1. Validasi Input yang Tidak Sesuai

Memasukkan karakter spesial, angka pada field yang seharusnya hanya menerima huruf, atau sebaliknya.

Contoh Request :

```
json
{
  "name": "John@Doe123"
}
```

Contoh Response :

```
json
{
  "status": 400,
  "error": "Nama hanya boleh berisi huruf."
}
```

2. Pengujian dengan Data Kosong atau Null

- Tidak mengisi input yang wajib diisi dan mengecek apakah sistem menampilkan pesan kesalahan yang sesuai.
- Memasukkan nilai null pada API request dan melihat respons dari sistem.

Contoh Request :

```
json
{
  "username": "",
  "password": "mypassword123"
}
```

Contoh

Response

:

```
json
{
  "status": 422,
  "error": "Username tidak boleh kosong."
}
```

Status Code : 422 Unprocessable Entity (Karena field wajib tidak diisi)

Contoh jika Null dalam API request :

```
json
{
  "email": null
}
```

Response :

```
json

{
  "status": 400,
  "error": "Email harus diisi."
}
```

Status code : 400 Bad Request

3. Pengujian Kombinasi Input yang Tidak Valid

- Menggunakan kombinasi input yang tidak masuk akal, seperti mengisi alamat dengan angka di semua kolom.
- Memasukkan dua format data berbeda dalam satu field (misalnya, nomor telepon dengan huruf).

Contoh Request (Alamat hanya angka) :

```
json

{
  "address": "123456789"
}
```

Response :

```
json

{
  "status": 400,
  "error": "Alamat tidak valid."
}
```

Status Code : 400 Bad Request

5.3. Standar Klasifikasi Tipe Pengujian Security

Pengujian security bertujuan untuk memastikan bahwa sistem atau API aman dari ancaman dan kerentanan. Selain memastikan kepatuhan terhadap kebijakan keamanan, QA dapat mengikuti beberapa panduan berikut dalam mengembangkan pengujian tipe security, diantaranya adalah:

1. Pengujian Autentikasi dan Otorisasi

- Pastikan sistem hanya mengizinkan akses kepada pengguna yang memiliki kredensial yang valid.

Contoh Request dengan kredensial salah :

```
json
{
  "username": "user123",
  "password": "passwordsalah"
}
```

Contoh Response :

```
json
{
  "status": 401,
  "error": "Username atau password salah."
}
```

Status Code: 401 Unauthorized

- Uji mekanisme login terhadap serangan brute-force dan credential stuffing.
Brute-force attack: Mencoba banyak kombinasi password hingga berhasil.
Credential stuffing: Menggunakan kredensial bocor dari situs lain.
Solusi: Implementasi mekanisme **rate-limiting** atau **captcha** setelah beberapa kali gagal login.

Contoh jika terlalu banyak percobaan login:

```
json
{
  "status": 429,
  "error": "Terlalu banyak percobaan login. Coba lagi dalam 15 menit."
}
```

Status Code : 429 Too Many Request

- Pastikan hak akses pengguna sesuai dengan peran yang diberikan (Role-Based Access Control).

Contoh Request (User biasa mencoba akses admin endpoint) :

```
pgsql

GET /admin/dashboard
Headers: Authorization: Bearer {user-token}
```

Contoh Response :

```
json

{
  "status": 403,
  "error": "Anda tidak memiliki izin untuk mengakses halaman ini."
}
```

Status Code : 403 Forbidden

2. Pengujian Validasi Input

- Cek apakah sistem memvalidasi input dengan benar untuk mencegah serangan seperti SQL Injection, Cross-Site Scripting (XSS), dan Command Injection.

Cek apakah sistem rentan terhadap SQL Injection

Contoh Request (Login dengan SQL Injection):

```
json

{
  "username": "' OR '1'='1",
  "password": "randompassword"
}
```

Jika API **tidak aman**, kemungkinan akan merespons dengan sukses login:

```
json

{
  "status": 200,
  "message": "Login berhasil"
}
```

Uji sistem terhadap serangan Cross-Site Scripting (XSS) :

Input script berbahaya dalam form komentar :

```
json
{
  "comment": "<script>alert('Hacked!')</script>"
}
```

Solusi: Gunakan output encoding agar script tidak dieksekusi.

- Gunakan teknik fuzzing untuk menguji seberapa baik sistem menangani input yang tidak valid atau berbahaya.

Menggunakan alat fuzzing seperti **AFL, Burp Intruder, atau ZAP** untuk mengirim input acak dan melihat bagaimana sistem bereaksi.

3. Pengujian Enkripsi dan Keamanan Data

- Pastikan data sensitif dienkripsi saat disimpan (data at rest) dan saat dikirim (data in transit).

Password tidak boleh disimpan dalam format plaintext:

```
json
{
  "password": "mypassword"
}
```

Solusi: Terapkan bcrypt atau Argon2 hashing

Contoh hashing yang benar :

```
json
{
  "password": "$2a$10$EIXmH9... (hashed)"
}
```


- Uji apakah komunikasi menggunakan protokol yang aman seperti TLS/SSL.

Cek apakah API hanya mengizinkan koneksi HTTPS

Gunakan alat seperti SSL Labs Test untuk mengevaluasi keamanan SSL/TLS.

- Periksa apakah sistem menyimpan kata sandi dengan hashing yang kuat

4. Pengujian Ketahanan terhadap Serangan Umum

- Gunakan alat seperti OWASP ZAP atau Burp Suite untuk mendeteksi kerentanan.

Jalankan **scan otomatis** dengan OWASP ZAP untuk mendeteksi celah keamanan umum.

- Lakukan penetration testing untuk mengevaluasi tingkat keamanan sistem secara menyeluruh.

Gunakan Metasploit atau Nmap untuk menguji kerentanan API dari sisi jaringan.

Coba eksploitasi endpoint yang kurang aman untuk melihat apakah sistem bisa diretas.

6. STANDAR PENGUJIAN BERBASIS SCENARIO

6.1. Prosedur Dasar

6.1.1. HTTP Methods: Mengirim API

HTTP Methods itu kayak gaya komunikasi seseorang ke API. Berikut beberapa API yang sering digunakan:

- 1) **GET** → Buat ngambil data. Contoh: “Mohon berikan daftar pengguna yang tersedia”
- 2) **POST** → Buat nambah data baru. Contoh: “Silakan tambahkan pengguna ini ke dalam sistem”
- 3) **PUT** → Buat update data *keseluruhan*. Contoh: “Harap perbarui seluruh informasi pengguna ini.”
- 4) **PATCH** → Buat update data *sebagian aja*. Contoh: “Cuman ganti nama user ini ya.”
- 5) **DELETE** → Buat hapus data. Contoh: “User ini sudah tidak diperlukan, silahkan hapus sistem”

Proses Testing:

- a. Pastikan API memberikan respons sesuai ekspektasi tiap metode.
- b. Cek jika data yang dikirim masuk ke database atau berubah sesuai permintaan.

6.1.2. HTTP Status Code: Response dari API

Status kode menunjukkan respons API terhadap permintaan yang dikirim, apakah berhasil, mengalami kesalahan dari sisi klien, atau terdapat masalah pada server.

- 1) **2xx (Success)** Menunjukkan bahwa permintaan berhasil diproses.
 - 200 OK** : Permintaan berhasil diproses tanpa kendala.
 - 201 Created** : Data baru berhasil dibuat dan ditambahkan ke dalam sistem.
- 2) **4xx (Client Error)** Terjadi kesalahan akibat permintaan yang tidak valid dari sisi klien.
 - 400 Bad Request** : Permintaan tidak dapat diproses karena format atau sintaks yang salah.
 - 401 Unauthorized** : Akses ditolak karena pengguna belum melakukan autentikasi.
 - 404 Not Found** : Sumber daya yang diminta tidak ditemukan pada server.
- 3) **5xx (Server Error)** Menunjukkan adanya kesalahan pada server saat memproses permintaan.
 - 500 Internal Server Error** : Terjadi kesalahan pada server yang menyebabkan permintaan tidak dapat diselesaikan.

Proses Testing:

- a. Pastikan API memberikan status code yang sesuai.
- b. Coba kirim request aneh-aneh buat lihat error handlingnya.

6.1.3. Authentication & Authorization: Siapa Saja Yang Boleh Masuk

Tidak semua mampu akses API karena terdapat 2 hal penting di sini:

- 1) **Authentication** (Autentikasi) Proses untuk memastikan identitas pengguna. Contohnya, penggunaan mekanisme seperti API Key, JWT Token, atau OAuth untuk memverifikasi bahwa pengguna adalah pihak yang berwenang mengakses sistem.
- 2) **Authorization** (Otorisasi) Proses untuk menentukan hak akses pengguna terhadap sumber daya tertentu. Contohnya, pengguna dengan peran biasa tidak memiliki izin untuk menghapus data milik administrator.

Proses Testing:

- a. Coba request tanpa token, harusnya ditolak.
- b. Coba pake token user biasa buat akses fitur admin, harusnya ga bisa.

6.1.4. Headers: Informasi Tambahan di Request

Headers tuh kayak embel-embel tambahan di surat yang seseorang kirim ke API.

- 1) **Content-Type** → Format data (application/json, text/plain, dll).
- 2) **Authorization** → Token untuk autentikasi.
- 3) **Accept-Language** → Bahasa yang diminta.

Proses Testing:

- a. Coba request tanpa header, apakah API tetap jalan?
- b. Kirim Content-Type yang salah, API harusnya nolak.

6.1.5. Path & Query Parameters: Memberikan Instruksi ke API

Ada dua cara untuk memberi detail tambahan ke API:

- 1) **Path Parameters** → Bagian dari URL, contoh:

GET /users/123

Artinya ambil user dengan ID 123.

- 2) **Query Parameters** → Tambahan di URL setelah? Contoh:

GET /users?role=admin&active=true

Artinya mengambil semua user yang role-nya admin dan masih aktif.

Proses Testing:

- a. Coba kirim request dengan parameter yang salah, harusnya API nolak.
- b. Pastikan API ngasih data yang sesuai sama parameter yang dikirim.

6.1.6. Versioning: API Jangan Sampai Bentrok!

Kadang API berkembang, tapi versi lama masih dipakai. Solusinya? Versioning.

- 1) Dapat menggunakan **Path**: /v1/users, /v2/users

- 2) Dapat menggunakan **Headers**: Accept: application/vnd.myapi.v2+json

Proses Testing:

- a. Pastikan versi API yang beda diberikan respons yang beda.
- b. Cek apakah API lama masih bisa jalan tanpa error.

6.1.7. **Pagination: Agar API Tidak Kelebihan Beban**

Terdapat **pagination** yang berguna untuk membatasi jumlah data yang diambil *per request*.

Contoh:

```
GET /users?page=2&limit=10
```

Artinya ambil **halaman kedua**, masing-masing **10 user** per halaman.

Proses Testing: Pastikan pagination bekerja sesuai permintaan (limit, page)

6.1.8. **Input Payload Validation: Data Masuk Harus Benar**

Pastikan API menerima data yang benar sebelum diproses.

Contoh input valid:

```
{
  "name": "Budi",
  "email": "budi@example.com"
}
```

Contoh input tidak valid:

```
{
  "name": "",
  "email": "budi@"
}
```

Harusnya API menolak data yang tidak valid:

Proses Testing:

- a. Kirim data yang kurang, API harus nolak.
- b. Coba input aneh-aneh (*script injection*, *SQL injection*), API harus aman.

6.1.9. **Response Payload Validation: Data Keluar Harus Benar**

API harus kasih data dengan format yang benar.

Contoh respons valid:

```
{
  "id": 1,
  "name": "Budi",
  "email": "budi@example.com"
}
```

Jika response yang diterima name tidak muncul atau email kosong, berarti ada yang salah.

Proses Testing:

- a. Cek apakah semua field yang dibutuhkan ada di respons API.
- b. Pastikan API tidak membocorkan data sensitif (misal password).

6.1.10. Error Handling: Supaya API Tidak Malfuntion

API yang baik harus kasih error yang jelas.

1) Error yang bagus :

```
{ "error": "User not found", "code": 404 }
```

2) Error yang jelek :

```
{ "message": "Something went wrong" }
```

Kenapa Error *"Something went wrong"* ini dianggap jelek? karena tidak informatif dan tidak membantu debugging.

Proses Testing:

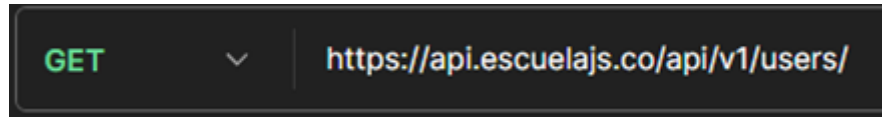
- a. Coba request dengan data salah, API harus kasih pesan error yang jelas.
- b. Pastikan API tidak kasih error 500 kalau cuma ada typo di request.

7. STANDAR PENGUJIAN MENGGUNAKAN POSTMAN

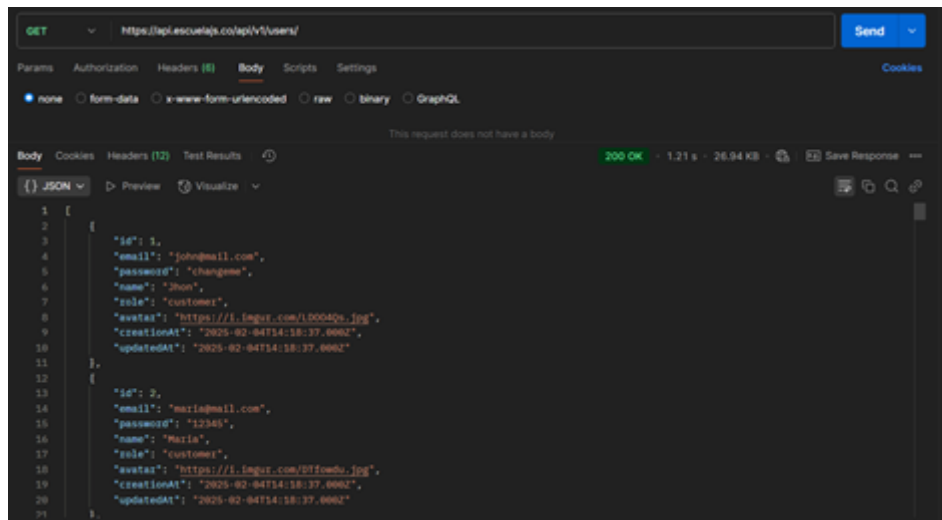
7.1. GET

Method Get digunakan untuk request data dan data pada server tidak akan berubah jika dilakukan get terus menerus. Berikut langkah-langkahnya :

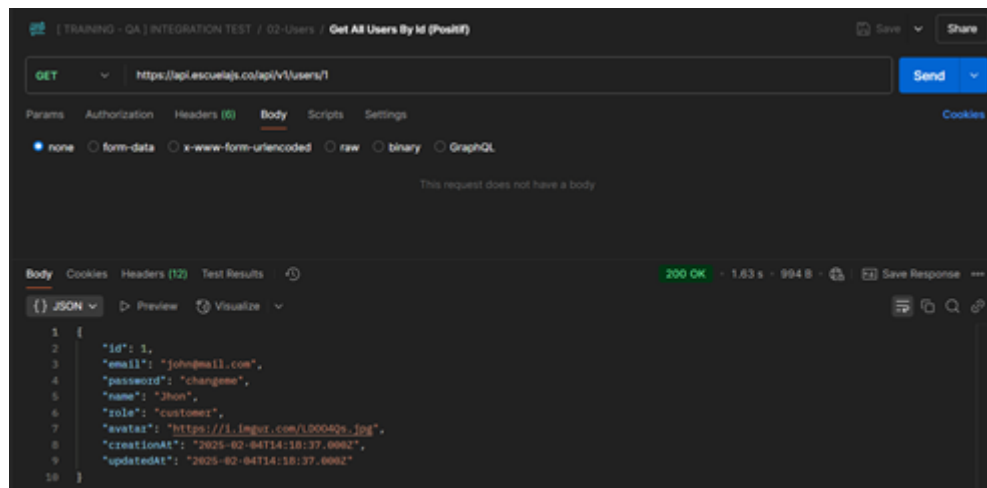
1. Buka aplikasi postman, kemudian buat request baru.
2. Masukkan URL API yang akan dihit, kemudian pastikan metode yang dipilih adalah GET.
Sebagai contoh, disini digunakan API untuk mendapatkan list user.



3. Jika URL sudah dipastikan benar dan metode yang dipilih adalah GET, klik tombol “Send” di sebelah kanan kolom URL.
4. Jika data yang diminta tersedia, hasil hit API akan muncul di tab “response” yang ada di bagian bawah.



5. Biasanya, metode GET jarang menggunakan parameter tambahan untuk filter data, jika ada parameter ini biasanya menjadi tambahan pada URL. Parameter ini biasanya digunakan untuk melihat detail dari sebuah data.

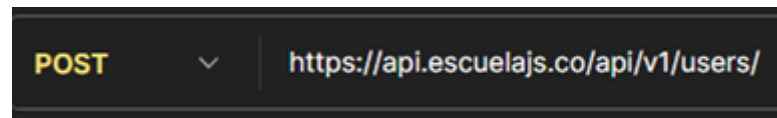


7.2. POST

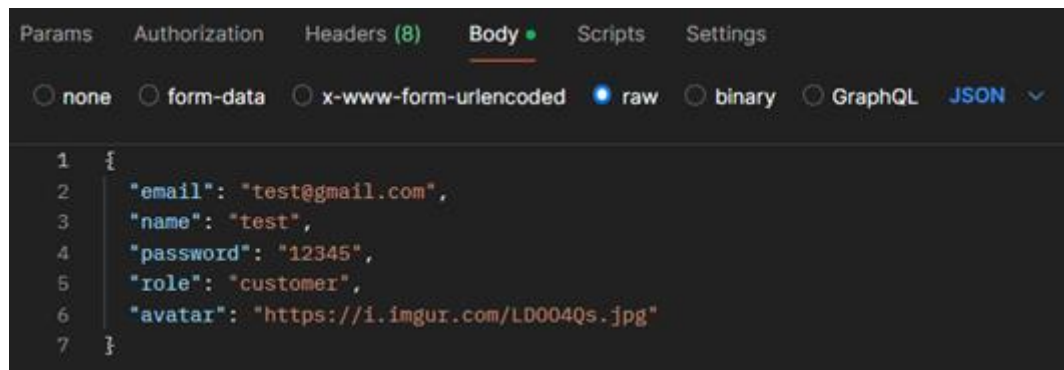
Method ini digunakan untuk mengirim data ke dalam server. Biasanya method ini digunakan apabila akan menambah data baru ke dalam server. Sehingga pada method post ini, ada data yang harus diisi sebagai inputan ke dalam server. Berikut langkah-langkahnya :

1. Buka aplikasi postman, kemudian buat request baru.

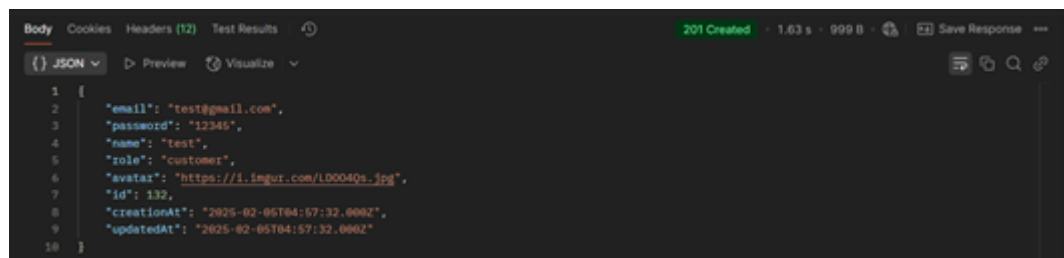
2. Masukkan URL API, kemudian pastikan method yang dipilih adalah POST.



3. Metode post ini memerlukan data yang akan dikirim ke server, sehingga data tersebut harus dilengkapi pada tab “body”. Data yang dikirim disesuaikan dengan yang akan diterima oleh server. Sebagai contoh disini, data yang dikirim berupa raw.



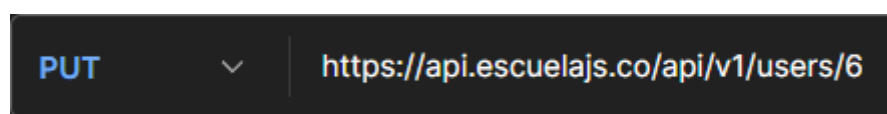
4. Jika sudah, klik tombol “send” seperti pada method GET.
5. Jika berhasil, pesan sukses akan ditampilkan pada tab “response”. Response juga akan menampilkan kode setelah hit API dilakukan, jika berhasil kode berawalan 2xx, jika gagal bisa kode 4xx yang berarti ada kesalahan dari inputan data atau kode 5xx jika terdapat error pada server. Disini kode yang dihasilkan adalah “201 Created”.



7.3. PUT

Method put digunakan untuk melakukan edit atau update data yang ada di database/server. Metode ini juga memerlukan inputan berupa data hasil update yang akan menggantikan data existing pada server atau database. Berikut langkah-langkahnya:

1. Buka aplikasi postman, kemudian buat request baru.
2. Masukkan URL API, kemudian pastikan method yang dipilih adalah PUT.



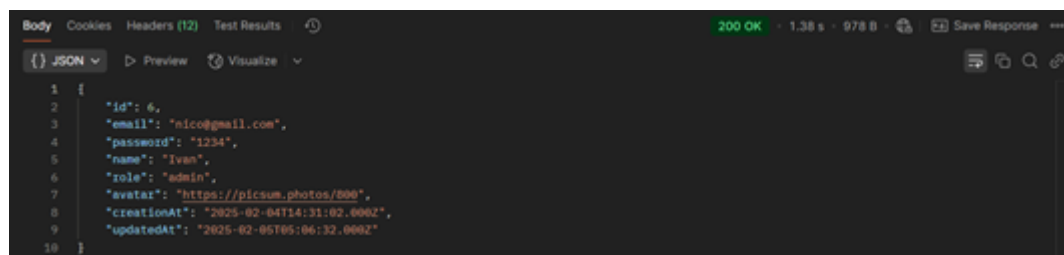
3. Masukkan data baru yang akan diupdate ke dalam server/database. Data bisa diinput pada tab “body” dengan format yang menyesuaikan. Disini data yang akan diupdate masih sama dengan method POST yaitu dengan format raw.

```

1  {
2      "email": "nico@gmail.com",
3      "password": "1234",
4      "name": "Ivan",
5      "role": "admin",
6      "avatar": "https://picsum.photos/800"
7  }

```

4. Klik “send” jika data sudah dipastikan benar.
5. Apabila berhasil, pesan akan ditampilkan pada tab response dengan ketentuan yang sama saat melakukan post. Pada hasil test kali ini, pesan yang ditampilkan adalah “200 OK” yang artinya data berhasil di update.



```

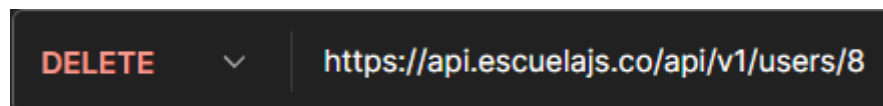
1  {
2      "id": 6,
3      "email": "nico@gmail.com",
4      "password": "1234",
5      "name": "Ivan",
6      "role": "admin",
7      "avatar": "https://picsum.photos/800",
8      "createdAt": "2025-02-04T14:31:02.000Z",
9      "updatedAt": "2025-02-05T05:06:32.000Z"
10 }

```

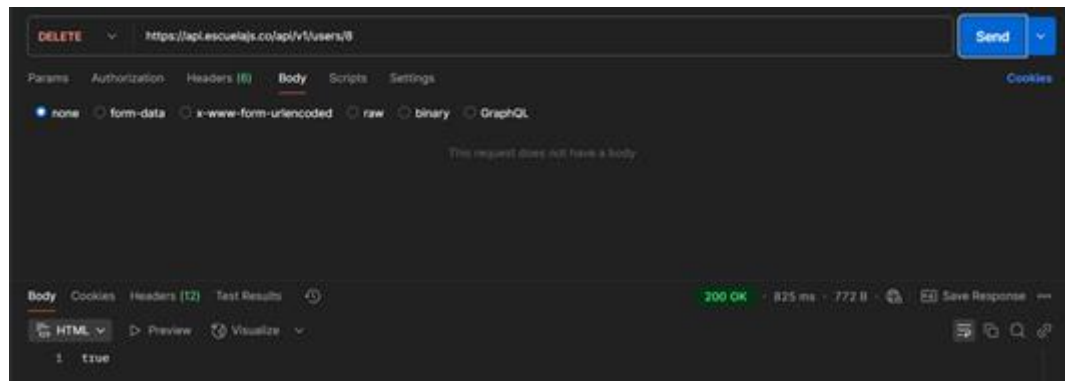
7.4. DELETE

Method ini sesuai namanya dilakukan untuk menghapus data yang ada di dalam database atau server. Adapun parameter data yang akan dihapus sama dengan metode GET, yaitu dengan menambahkan parameter (biasanya id) pada URL. Jika tidak menggunakan parameter, ada kemungkinan semua data pada server akan terhapus karena data yang akan dihapus tidak spesifik. Berikut langkah-langkahnya :

1. Buka aplikasi postman, kemudian buat request baru.
2. Masukkan URL API, kemudian pastikan method yang dipilih adalah DELETE.



3. Pastikan data dengan id tersebut ada pada database. Klik “send” untuk melakukan eksekusi.
4. Jika proses delete berhasil, pesan akan ditampilkan pada tab response dengan code “200 OK”



8. STANDAR PENGUJIAN AUTOMATION MENGGUNAKAN NEWMAN

Berikut tahapan-tahapan yang perlu diikuti untuk menggunakan newman :

8.1. Install Newman

1. Untuk menggunakan Newman, yang pertama kali dilakukan ialah menginstal Node.js terlebih dahulu, karena Newman berjalan di atas Node.js. Jika belum terinstal, kunjungi situs resmi Node.js (<https://nodejs.org>) dan unduh installer yang sesuai
2. Verifikasi Instalasi Node.js:

Setelah Node.js terinstal, buka terminal atau command prompt, lalu jalankan perintah berikut:

```
1 node -v
```

Perintah ini digunakan untuk menampilkan versi Node.js yang terpasang pada sistem. Jika versi muncul, berarti instalasi Node.js berhasil.



```
MINGW64:/c
HP@MUNEZ MINGW64 /c
$ node -v
v20.18.0
HP@MUNEZ MINGW64 /c
$ |
```

3. Instal Newman:

Setelah Node.js terinstal, ketikkan command berikut untuk menginstal Newman :

```
1 npm install -g newman
```

Perintah ini akan mengunduh dan menginstal versi terbaru Newman secara global di perangkat. Bendera -g digunakan agar Newman diinstal sebagai paket global, sehingga bisa diakses dari direktori mana pun di komputer. Setelah semua langkah ini selesai, Newman siap digunakan untuk menjalankan koleksi API dari Postman.

8.2. SetUp Newman: Reporting dengan Allure

1. Export collection

Export Collection dari Postman sebagai file JSON. Export sebagai Collection v2.1

2. Buatlah folder dengan struktur didalamnya seperti gambar berikut :

```
project-root/
├─ node_modules/
├─ allure-results/
├─ allure-report/
├─ .env
├─ run-newman.js
├─ Ecommerce-Development.postman_collection.json
├─ Ecommerce-development.postman_environment.json
├─ package.json
└─ package-lock.json
```

Fungsi dari struktur tersebut ialah:

- **node_modules/**

Fungsi: Folder ini berisi dependensi atau library yang diinstal menggunakan npm (Node Package Manager).

Sumber: Dihasilkan secara otomatis setelah menjalankan perintah npm install.

- **allure-results/**

Fungsi: Folder ini berisi hasil pengujian yang dapat digunakan untuk menghasilkan laporan Allure. File di dalamnya berupa file JSON atau XML yang mendokumentasikan hasil setiap pengujian.

Sumber: Dihasilkan oleh Newman saat dijalankan dengan reporter Allure (--reporter-allure-export allure-results/).

- **allure-report/**

Fungsi: Folder ini berisi laporan Allure dalam format HTML yang dihasilkan berdasarkan data di allure-results.

Sumber: Dihasilkan setelah menjalankan perintah allure generate.

- **.env**

Fungsi: File ini digunakan untuk menyimpan variabel lingkungan (environment variables), seperti URL, API key, atau pengaturan lainnya yang bersifat rahasia dan spesifik untuk lingkungan pengujian.

Sumber: Dibuat secara manual.

- **run-newman.js**

Fungsi: Script Node.js yang digunakan untuk menjalankan Newman secara programatik dengan parameter tertentu, seperti koleksi Postman, environment, atau reporter.

Sumber: Dibuat secara manual oleh pengguna.

- **Ecommerce-Development.postman_collection.json**

Fungsi: File koleksi Postman yang berisi kumpulan request API yang akan diuji.

Sumber: Diekspor dari Postman.

- **Ecommerce-development.postman_environment.json**

Fungsi: File environment Postman yang berisi variabel-variabel yang digunakan untuk menjalankan koleksi, seperti URL base API, token, atau parameter lainnya.

Sumber: Diekspor dari Postman.

- **package.json**

Fungsi: File konfigurasi untuk proyek Node.js yang berisi metadata proyek, dependensi, dan script yang dapat dijalankan.

Sumber: Dihasilkan oleh perintah npm init dan diperbarui secara manual atau melalui instalasi paket.

- **package-lock.json**

Fungsi: File ini mencatat versi spesifik dari semua dependensi yang diinstal untuk memastikan konsistensi penginstalan di berbagai lingkungan.

Sumber: Dihasilkan secara otomatis saat menjalankan perintah npm install.

Dan didalam collection struktur file harus mengikuti gambar berikut :

```
TESTING_QA-PORTAL_LOGIN-DEVELOPMENT
├─ manage-user
│   ├── positif
│   │   ├── get-id-user
│   │   └─ post-data-user
│   └─ negatif
│       ├── get-id-user-wrong-request
│       └─ post-data-user-wrong-data
```

Untuk scenario positif dan negative dari tiap – tiap test dipisahkan. Dan fungsi dari pemisahan positif dan negatif sendiri bertujuan untuk **Memastikan Validitas Fungsionalitas**

Skenario Positif: Menguji apakah sistem berfungsi sesuai dengan spesifikasi atau harapan dalam kondisi normal. Contohnya adalah memasukkan data yang valid untuk mendapatkan hasil yang diharapkan.

Skenario Negatif: Menguji ketahanan sistem saat dihadapkan pada kondisi tidak valid, seperti input data yang salah atau skenario di luar ekspektasi.

3. Instalasi newman dengan report allure, gunakan command berikut untuk installasi newman dengan allure report:

```
npm install newman newman-reporter-allure --save-dev
```

4. Instalasi Java :

- Kunjungi AdoptOpenJDK atau OpenJDK.
- Pilih versi yang diinginkan (misalnya, OpenJDK 11).
- Unduh installer dan ikuti instruksi untuk menginstalnya.
- Masukkan pada PATH system variable pada environment variable komputer lokal

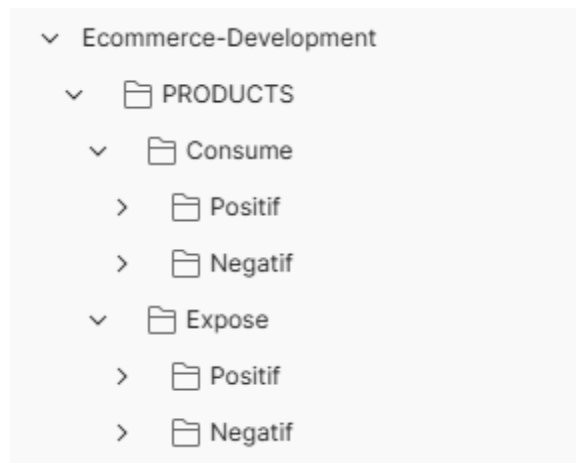
```
;%JAVA_HOME%\bin
```

- Konfirmasi versi java

```
C:\Users\user>java -version
java version "21.0.5" 2024-10-15 LTS
Java(TM) SE Runtime Environment (build 21.0.5+9-LTS-239)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.5+9-LTS-239, mixed mode, sharing)
```

8.3. Implementation

8.3.1. Membuat Collection Sesuai Standar



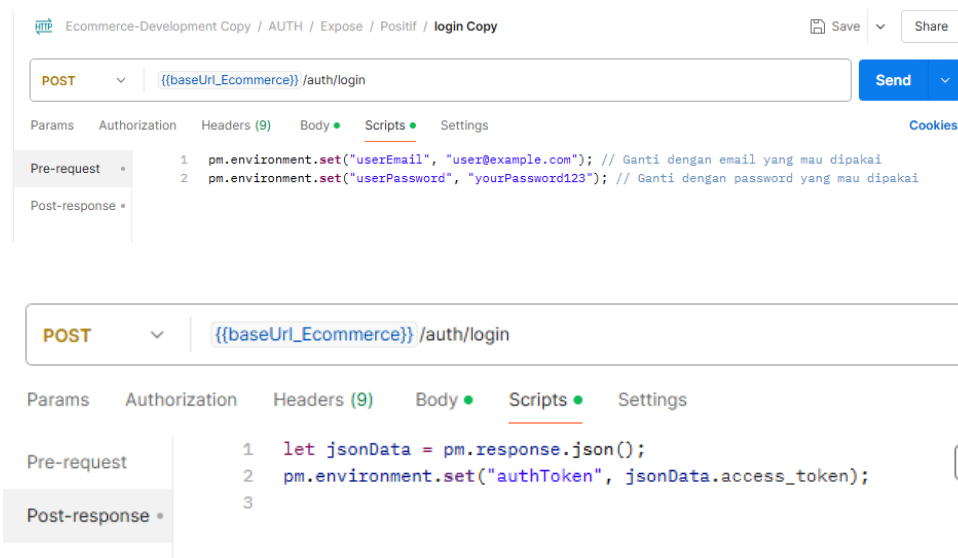
Pastikan pada struktur collection sudah sama dengan gambar yang ada diatas.

	Variable	Type	Initial value	Current value	...
☒	baseUrl_Ecommerce	default	https://api.escuelajs.co/api/v1	https://api.escuelajs.co/api/v1	
☒	authToken	default			
☒	userEmail	default		newman1@newman.com	
☒	userPassword	default		newmanAdmin	

Dan untuk kondisi API yang membutuhkan auth token, berikut langkah yang harus diikuti:

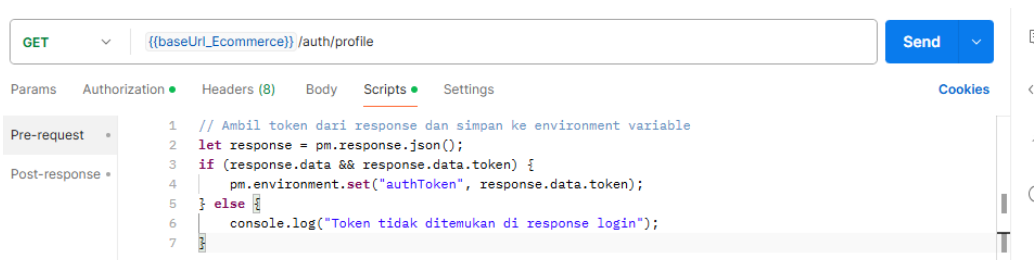
1. Login API untuk Mendapatkan Token:

- Endpoint POST `{{baseUrl_Ecommerce}}/auth/login` digunakan untuk melakukan autentikasi user. Setelah login berhasil, response dari API akan berisi token autentikasi (contoh: `access_token`).
- Token ini disimpan ke dalam environment menggunakan Postman script berikut, token ini digunakan sebagai authorization pada API berikutnya :



2. Menguji API yang Membutuhkan Auth Token:

- Endpoint GET `{{baseUrl_Ecommerce}}/auth/profile` membutuhkan header authorization dengan nilai Bearer `<authToken>`. Token ini diambil dari environment yang telah di-set sebelumnya.
- Postman script berikut digunakan untuk mengambil token dari response dan menyimpannya ke environment:
- Menyimpan authToken setelah login berhasil



- Postman script berikut digunakan untuk memverifikasi response:

```
1 pm.test("Response status is 200", function () {  
2   pm.response.to.have.status(200);  
3 });
```

Gambar 1 Cek status response 200

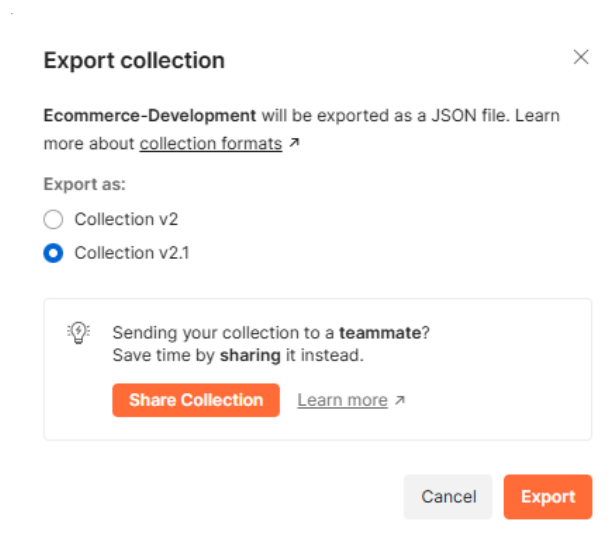
```
5 pm.test("User ID is not null", function () {  
6   let jsonData = pm.response.json();  
7   pm.expect(jsonData.id).to.not.be.null;  
8 });
```

Gambar 2 Cek User ID Tidak Null

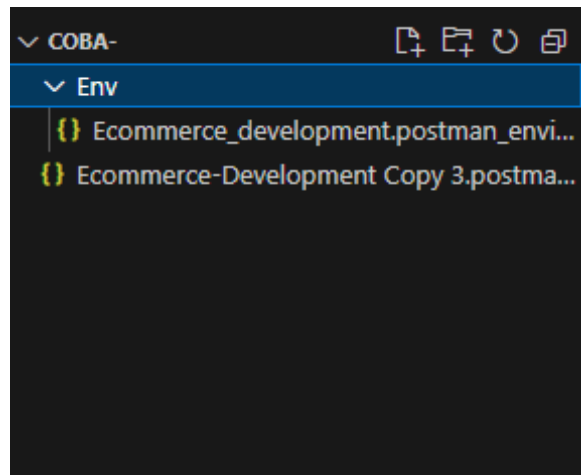
```
10 pm.test("User name is correct", function () {  
11   let jsonData = pm.response.json();  
12   pm.expect(jsonData.name).to.eql("Testing_Newman");  
13 });
```

Gambar 3 Cek Username Yang Benar

Dan simpan data sensitif di environment, seperti url, username, dan password. Selanjutnya export collection dan env yang digunakan. Pilih export collection v2.1

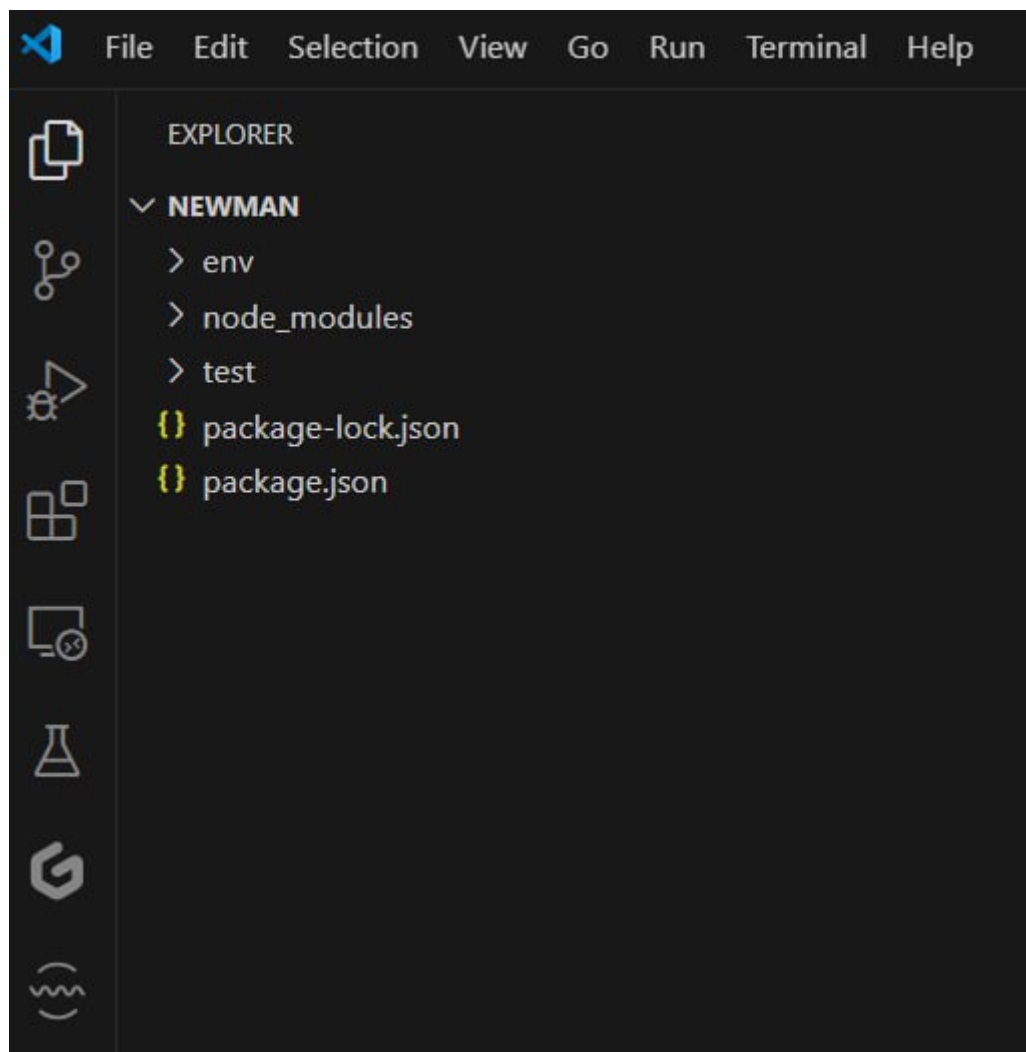


Setelah itu buat folder baru dengan berisi file collection tadi dan folder env, lalu buka Vsode



Ketika sudah membuka vscode, install newman dengan command berikut :

```
npx newman neuron init
```



Ketika sudah diinstal nanti akan muncul node_modules dan package.jsonnya. npx newmanneuron tersebut ialah npm custom dari tim SQ yang akan berfungsi sebagai instalasi newman beserta dependency lainnya.

8.4. Execution

8.4.1. Menjalankan Report Newman di HTML

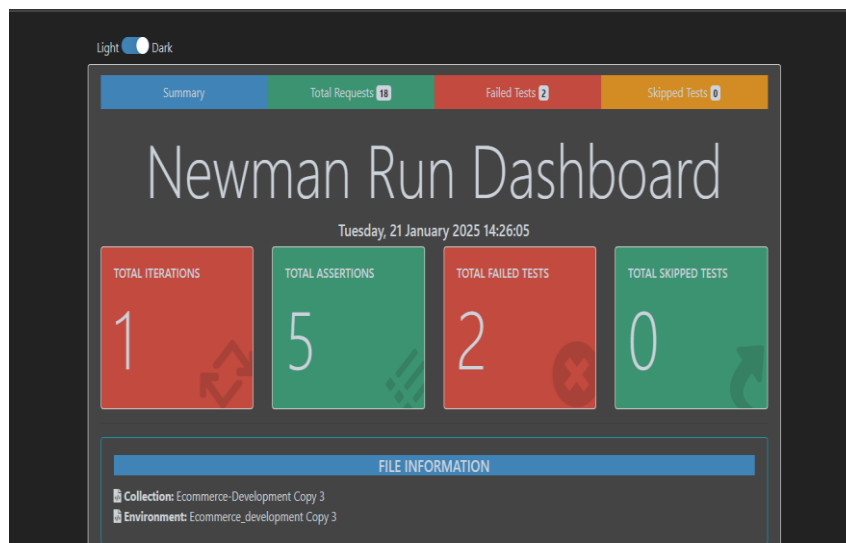
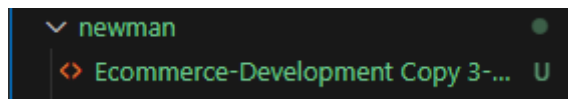
Sebelum di run tambahkan script berikut di file package.json :

```
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1",
  "test-html": "newman run <collection_name>.postman_collection.json -e <environment_name>.postman_environment.json -r html"
}
```

Untuk run dengan hasil html cukup jalan kan dengan command berikut :

```
> newman run <Collection_name>.json/ -e <Env_name>.json -r html
```

Dan hasilnya akan tergenerate seperti ini :



8.4.2. Menjalankan Newman di Terminal

Untuk menjalankan melalui terminal cukup jalankan command seperti berikut :

```
PS C:\Users\user\qa-training-project2> newman run <collection_file>.postman_collection.json -e <environment_file>.postman_environment.json --reporters cli
```

Hasilnya akan seperti berikut :

```

❏ negatif
└─ callback status
  POST https://myindibiz-dev.apps.kpaasjtn1.telkom.co.id/api/callback-status [200 OK, 317B, 146ms]

└─ venue lists
  POST https://myindibiz-dev.apps.kpaasjtn1.telkom.co.id/api/venue-lists [200 OK, 315B, 116ms]

└─ signout partner
  POST [errored]
    runtime:extensions-request: request url is empty

└─ profile user
  POST https://myindibiz-dev.apps.kpaasjtn1.telkom.co.id/api/profile-user [200 OK, 331B, 122ms]

└─ hitung ppn
  POST https://myindibiz-dev.apps.kpaasjtn1.telkom.co.id/api/hitung-ppn [200 OK, 382B, 75ms]

└─ user signin
  POST https://myindibiz-dev.apps.kpaasjtn1.telkom.co.id/api/user-signin [200 OK, 356B, 70ms]

└─ generate link forgot password
  POST https://myindibiz-dev.apps.kpaasjtn1.telkom.co.id/api/generate-link-forgot-password [200 OK, 347B, 75ms]

```

	executed	failed
iterations	1	0
requests	14	2
test-scripts	0	0
prerequest-scripts	0	0
assertions	0	0
total run duration: 5.1s		

8.4.3. Menjalankan Newman di Allure

Tahap pertama install allure comandline dengan command berikut :

```
npm install -g allure-commandline
```

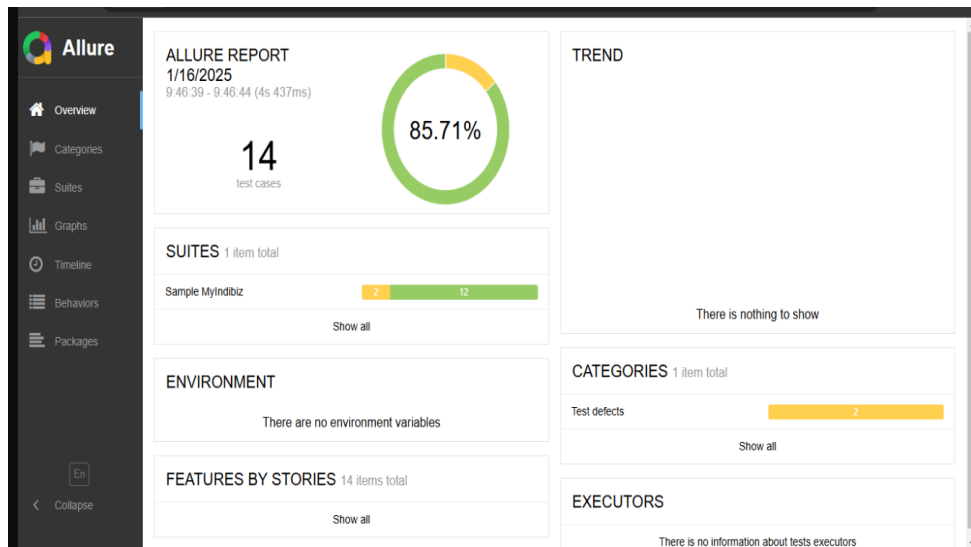
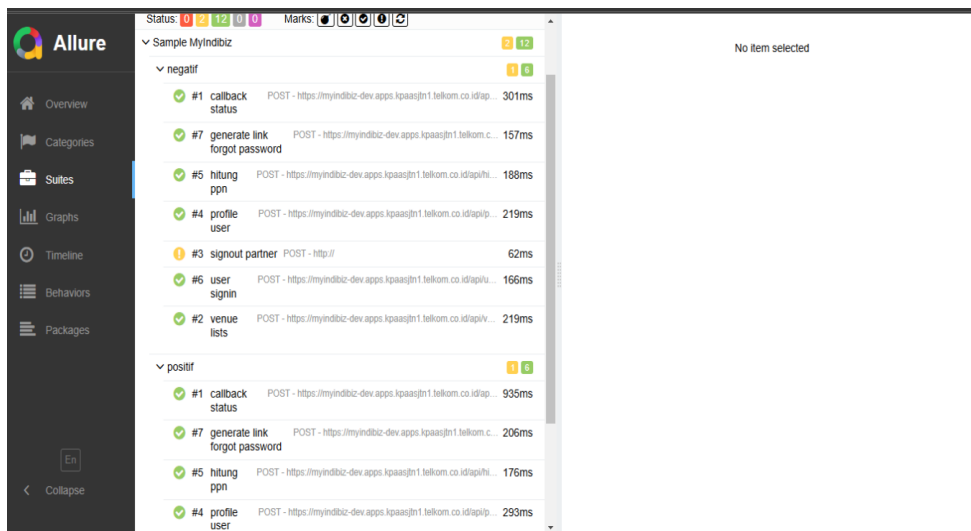
Lalu ketikan command berikut, yang berguna untuk menghasilkan laporan Allure dari hasil pengujian yang tersimpan di folder allure-results :

```
allure generate allure-results --clean -o allure-report
```

Dan yang terakhir gunakan command Berikut untuk langsung membuka laporan allure:

```
allure open allure-report
```

Hasilnya akan seperti berikut :

Status: 0 2 12 10 0 Marks: 0 0 0 0 0

Sample MyIndibiz 2 12

negatif 0 1 8

Item	Result	URL	Duration
#1 callback status	Success	POST - https://myindibiz-dev.apps.kpaasjtn1.telkom.co.id/api...	301ms
#7 generate link forgot password	Success	POST - https://myindibiz-dev.apps.kpaasjtn1.telkom.c...	157ms
#5 hitung ppn	Success	POST - https://myindibiz-dev.apps.kpaasjtn1.telkom.co.id/api/h...	188ms
#4 profile user	Success	POST - https://myindibiz-dev.apps.kpaasjtn1.telkom.co.id/api/p...	219ms
#3 signout partner	Failure	POST - http://	62ms
#6 user signin	Success	POST - https://myindibiz-dev.apps.kpaasjtn1.telkom.co.id/api/u...	166ms
#2 venue lists	Success	POST - https://myindibiz-dev.apps.kpaasjtn1.telkom.co.id/api/v...	219ms

positif 0 1 8

Item	Result	URL	Duration
#1 callback status	Success	POST - https://myindibiz-dev.apps.kpaasjtn1.telkom.co.id/api...	935ms
#7 generate link forgot password	Success	POST - https://myindibiz-dev.apps.kpaasjtn1.telkom.c...	206ms
#5 hitung ppn	Success	POST - https://myindibiz-dev.apps.kpaasjtn1.telkom.co.id/api/h...	176ms
#4 profile user	Success	POST - https://myindibiz-dev.apps.kpaasjtn1.telkom.co.id/api/p...	293ms

No item selected